

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and

system performance (BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

I Vishali P (192542313) certify that this submission is my original work and

that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Vishali P

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock. Suggest an optimisation to eliminate the deadlock without changing process logic.

1. Given Code

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
```

unlock(R1)

Process P2:

lock(R2)

wait(100 ms)

lock(R1)

critical section

unlock(R1)

unlock(R2)

2. Execution Trace

Initially, both resources are free.

Time	P1	P2
0 ms	Locks R1	Locks R2
100 ms	Requests R2	Requests R1
After 100 ms	Waits for R2	Waits for R1

Now:

- **R1 is held by P1.**
- **R2 is held by P2.**
- P1 is waiting for R2, which P2 holds.
- P2 is waiting for R1, which P1 holds.

Therefore, neither process can continue.

3. Exact Condition Causing Deadlock

The deadlock occurs because of **circular waiting**:

P1 → holds R1 → waits for R2

↓

P2 → holds R2 → waits for R1

This creates a circular dependency:

P1 → R2 → P2 → R1 → P1

The wait(100 ms) increases the chance of deadlock by giving both processes time to acquire their first resource before requesting the second one.

4. Deadlock Conditions

The four necessary conditions for deadlock are satisfied:

1. **Mutual Exclusion** – Only one process can hold a resource at a time.
2. **Hold and Wait** – P1 holds R1 while waiting for R2; P2 holds R2 while waiting for R1.
3. **No Preemption** – A resource cannot be forcibly taken from a process.
4. **Circular Wait** – P1 waits for P2's resource and P2 waits for P1's resource.

Hence, deadlock occurs.

5. Optimization to Eliminate Deadlock

Use a **consistent resource-locking order** in both processes.

For example:

Process P1:

lock(R1)

wait(100 ms)

lock(R2)

critical section

unlock(R2)

unlock(R1)

Process P2:

lock(R1)

wait(100 ms)

lock(R2)

critical section

unlock(R2)

unlock(R1)

Now both processes acquire resources in the order:

R1 → R2

If P1 has R1, P2 must wait for R1 before it can acquire any resource. Therefore, P2 cannot hold R2 while waiting for R1.

6. Why This Works

The consistent locking order **breaks the circular-wait condition**.

P1: R1 → R2

P2: R1 → R2

There is no possibility of:

P1: R1 → waits for R2

P2: R2 → waits for R1

Therefore, deadlock is prevented.

7. Conclusion

The exact cause of the deadlock is the **opposite order of resource acquisition**: P1 locks **R1 then R2**, while P2 locks **R2 then R1**. This satisfies all four deadlock conditions and creates a circular wait. The simplest optimization is to enforce a **global and consistent resource-locking order**, such as **R1 → R2**, in both processes. This eliminates deadlock without changing the actual critical-section logic.

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready. You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ... Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

1. Problem

The system has **only 30% CPU utilisation**, even though processes are ready to execute.

The scheduler log shows frequent transitions:

Ready → Waiting → Ready → Waiting → Ready

This indicates that processes are repeatedly being blocked and awakened instead of continuously using the CPU.

2. Debugging the Cause

A process normally follows:

Ready → Running → Waiting → Ready

The **Waiting state** occurs when a process waits for an event such as:

- I/O operation
- Disk access
- Network response
- Lock/semaphore
- Resource availability

In this case, frequent **Ready → Waiting** transitions suggest that processes are spending too much time waiting for I/O or synchronization resources.

3. Possible Cause

A likely cause is **excessive I/O or synchronization blocking**.

For example:

Process becomes Ready



Gets CPU



Requests I/O / resource



Moves to Waiting



I/O completes



Becomes Ready again



Gets CPU



Requests I/O again

Because the process spends most of its time waiting, the CPU remains idle.

4. Effect on CPU Utilisation

If many processes are waiting simultaneously:

CPU → Idle

Processes → Waiting

Even though processes exist in the system, they are not performing enough CPU-intensive work.

This results in:

- Low CPU utilisation

- Increased waiting time
- More context switching
- Lower throughput
- Poor overall system performance

5. System-Level Optimisation

The main optimisation is to **reduce unnecessary blocking and improve I/O handling**.

Possible solutions include:

1. Use **asynchronous/non-blocking I/O** so processes do not remain blocked unnecessarily.
2. **Increase I/O buffering** to reduce frequent I/O requests.
3. **Reduce unnecessary synchronization** such as excessive lock or semaphore operations.
4. **Improve scheduling** so another ready process can use the CPU while one process waits.
5. Use **appropriate time slices** to reduce excessive context switching.

6. Recommended Solution

The best system-level approach is to use **asynchronous I/O with efficient scheduling**.

Instead of:

Running → Waiting → Ready

the process can initiate an I/O operation and allow another ready process to execute.

Process P1 → I/O request

↓

Continue/another process

↓

CPU remains busy

This keeps the CPU busy while I/O operations are performed.

7. Expected Result

After optimisation:

CPU Utilisation

Before → 30%

After → Higher utilisation

The exact improvement depends on the workload, but reducing blocking and context-switching overhead can significantly improve CPU utilisation.

8. Conclusion

The low CPU utilisation is mainly caused by **frequent process blocking due to I/O or synchronization**, causing repeated **Ready → Waiting → Ready** transitions. The system should reduce unnecessary blocking using **asynchronous I/O, buffering, efficient synchronization, and proper CPU scheduling**. This allows the CPU to execute other ready processes instead of remaining idle, thereby improving **CPU utilisation and system throughput**.

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory

map shows: ● Code: 5 MB

● Heap: 150 MB

● Stack: 1 MB per thread ● 25 threads running ● Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

1. Given Information

- Code = **5 MB**
- Heap = **150 MB**
- Stack = **1 MB per thread**
- Number of threads = **25**
- Physical RAM = **2 GB**

2. Memory Calculation

Memory used by thread stacks:

$$25 \times 1 \text{ MB} = 25 \text{ MB}$$

Approximate total memory:

Code + Heap + Thread Stacks

$$\begin{aligned} &= \quad 5 \quad \text{MB} \quad + \quad 150 \quad \text{MB} \quad + \quad 25 \quad \text{MB} \\ &= \mathbf{180 \text{ MB}} \end{aligned}$$

So, the given memory usage is only about **180 MB**, which is much smaller than the available **2 GB RAM**.

3. Identify the Memory Issue

Since physical RAM is not exhausted, the frequent page faults are **not primarily caused by total memory capacity**.

The likely issue is **poor memory locality / excessive virtual-memory activity**.

In a multi-threaded application, threads may frequently access different parts of the large **150 MB heap**. If the working set does not fit efficiently in the CPU cache/TLB, pages may be repeatedly loaded into memory, causing **page faults**.

Another possibility is **memory fragmentation**, where the heap is divided into many small, scattered allocations. This reduces locality and increases memory-management overhead.

4. Why Page Faults Reduce Performance

When a required page is not currently available in physical memory, a page fault occurs.

The operating system must:

1. Pause the thread.
2. Locate the required page.
3. Load the page into physical memory.
4. Update the page table.
5. Resume execution.

If this happens frequently, threads spend more time waiting for memory than executing instructions.

5. Optimisation 1 – Improve Memory Locality

Organise data so that frequently accessed data is stored **close together**.

For example:

- Use contiguous arrays instead of scattered allocations.
- Reduce unnecessary pointer-based structures.
- Access data sequentially.
- Reduce random access to the large heap.

This improves **spatial and temporal locality**, reducing page faults and cache/TLB misses.

6. Optimisation 2 – Reduce Memory and Thread Overhead

Reduce unnecessary memory usage by:

- Reducing the number of threads if all 25 are not required.
- Reusing memory instead of repeatedly allocating/freeing objects.
- Using memory pools.
- Reducing unnecessary heap allocations.

For example, instead of creating many temporary objects:

Allocate → Use → Free

Allocate → Use → Free

use a memory pool:

Allocate Pool → Reuse → Reuse → Reuse

This reduces heap fragmentation and memory-management overhead.

7. Expected Result

After optimisation:

Better memory locality



Fewer page faults



Less waiting for memory



Higher CPU utilisation



Better program performance

8. Conclusion

The calculated memory usage is approximately **180 MB**, which is far below the **2 GB physical RAM**. Therefore, simple RAM exhaustion is not the main problem. The frequent page faults are more likely due to **poor memory locality, scattered heap access, fragmentation, or an inefficient working set**.

Two main optimisations are:

1. **Improve memory locality** by using contiguous and sequential data structures.
2. **Reduce memory-management overhead** using fewer unnecessary threads, memory pooling, and reduced heap allocations.

These optimisations can reduce page faults and significantly improve the performance of the multi-threaded program.

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler: • Q1 (Highest Priority): Interactive • Q2: System

• Q3: Batch • Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed. Debug the root cause of starvation and suggest an optimisation to ensure fairness.

1. Given Information

The system uses a **Multilevel Queue Scheduling** algorithm:

- **Q1 – Highest Priority:** Interactive processes, Time Quantum = 5 ms
- **Q2 – System processes:** Time Quantum = 10 ms
- **Q3 – Lowest Priority:** Batch processes, FCFS
- **P5:** Located in Q3
- **Problem:** P5 never gets CPU time.

2. Root Cause of Starvation

The main cause is **strict priority scheduling between the queues**.

The scheduler always gives preference to higher-priority queues:

Q1 → Q2 → Q3

If Q1 continuously contains ready processes, the scheduler keeps executing Q1 processes. Q3 is considered only when the higher-priority queues are empty.

For example:

Q1 → P1 → P2 → P3 → P4 → ...

↓

Q2 → P6 → P7 → ...

↓

Q3 → P5

Since new processes continuously arrive in Q1 or Q2, **P5 remains waiting in Q3 indefinitely.**

3. Why P5 Experiences Starvation

P5 is a **batch process**, so it has the lowest priority.

Even though P5 is ready, the scheduler selects processes from Q1 and Q2 first.

This creates:

P5 Ready → Waiting → Waiting → Waiting...

P5 may never receive CPU time.

This condition is called **starvation (indefinite blocking)**.

4. Important Observation

The **FCFS policy inside Q3 is not the main problem.**

FCFS only determines the order of processes **within Q3**. The real problem is that the scheduler may not reach Q3 at all because Q1 and Q2 continuously receive CPU time.

5. Optimisation – Aging

The best solution is **aging**.

In aging, the priority of a process is gradually increased as it waits for a long time.

For example:

P5 starts → Q3

↓

Waits for a long time

↓

Priority gradually increases

↓

Moved/promoted to Q2

↓

Gets CPU time

Eventually, P5 gets a higher priority and cannot be postponed indefinitely.

6. Alternative Optimisation – CPU Time Allocation

Another solution is to reserve a fixed percentage of CPU time for each queue.

For example:

Q1 → 50% CPU

Q2 → 30% CPU

Q3 → 20% CPU

Even when Q1 and Q2 have continuous processes, Q3 receives its guaranteed CPU share.

7. Fairness Improvement

A practical solution is to combine **aging with periodic priority adjustment**:

- Increase the priority of processes that wait too long.
- Give Q3 a guaranteed CPU time.
- Prevent indefinite postponement.

8. Result

Before optimisation:

Q1 → Q1 → Q1 → Q2 → Q1 → Q1 → Q2 → ...

↓

P5

Starved

After aging:

P5 waits → Priority increases → P5 promoted → CPU allocated

9. Conclusion

P5 suffers from **starvation** because **Q3 has the lowest priority and strict priority scheduling continuously favours Q1 and Q2**. The FCFS policy of Q3 does not solve this because the scheduler may never reach Q3.

The recommended optimisation is **aging**, where the priority of P5 increases as its waiting time increases. A **minimum CPU allocation for Q3** can also be used to guarantee fairness and ensure that every process eventually gets CPU time.

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead. Log details show: ● Average burst time per thread: 2ms ● I/O wait frequency: very high ● Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

1. Given Information

- Context switches = **20,000 switches/second**
- Average CPU burst = **2 ms**
- I/O wait frequency = **Very high**
- Scheduling = **Preemptive**
- Time quantum = **1 ms**

2. Why Context Switching Is Very High

The main reason is that the **time quantum (1 ms) is smaller than the average CPU burst time (2 ms)**.

A thread normally needs about **2 ms** of CPU time, but the scheduler interrupts it after only **1 ms**.

Therefore:

Run 1 ms → Context Switch → Run 1 ms → Context Switch

The thread is repeatedly preempted before completing its CPU burst.

3. Effect of High I/O Wait

The **very high I/O wait frequency** also increases context switching.

When a thread requests I/O:

Running → Waiting

The scheduler immediately selects another ready thread:

Thread A → I/O Wait



Context Switch



Thread B → Running

When Thread A's I/O completes, it becomes ready again and may require another context switch.

4. Main Causes

There are two major causes:

1. **Very small time quantum (1 ms)** compared with the 2 ms average CPU burst.
2. **Frequent I/O blocking and waking**, causing repeated scheduling decisions and context switches.

As a result, a large amount of CPU time is spent on **context-switch overhead instead of useful computation**.

5. Optimisation 1 – Increase Time Quantum

Increase the time quantum from **1 ms** to a value closer to the average CPU burst, such as **2–4 ms**, depending on workload.

For example:

Before:

Quantum = 1 ms

Burst = 2 ms

After:

Quantum = 2–4 ms

Burst = 2 ms

This allows a thread to complete more of its CPU burst before being preempted.

Result:

Fewer preemptions



Fewer context switches



Lower overhead



Better CPU performance

6. Optimisation 2 – Reduce I/O Blocking

The second optimisation is to reduce unnecessary I/O waits.

Methods include:

- Use **asynchronous/non-blocking I/O**.
- Use **I/O buffering**.
- Batch multiple I/O operations.
- Avoid unnecessary blocking system calls.
- Use efficient I/O scheduling.

This allows threads to spend more time performing useful CPU work instead of repeatedly moving between **Running** and **Waiting** states.

7. Before Optimisation

Thread A



Run 1 ms



Preempt



Context Switch



Thread B



Run 1 ms



Preempt



Context Switch

Frequent I/O makes the switching even worse.

8. After Optimisation

Thread A



Run 2–4 ms



Complete CPU burst



I/O if required



Thread B

This reduces unnecessary preemptions.

9. Expected Result

Factor	Before	After
Time Quantum	1 ms	2–4 ms
Context Switches	Very high	Lower
I/O Blocking	Very high	Reduced
CPU Overhead	High	Lower
Performance	Poor	Improved

10. Conclusion

The **20,000 context switches/sec** are mainly caused by the **1 ms time quantum being smaller than the 2 ms average CPU burst**, combined with **very frequent I/O waits**. The two major optimisations are **increasing the time quantum** to reduce unnecessary preemption and **reducing I/O blocking** using asynchronous I/O, buffering, and batching. These changes reduce context-switch overhead and improve overall server performance.

Q6. Semaphore Synchronization Debugging

A producer-consumer application occasionally hangs during execution. The semaphore values observed are:

mutex = 1

empty = 0

full = 10

Buffer size = 10

Producer Code:

wait(mutex)

wait(empty)

produce item

signal(full)

signal(mutex)

Consumer Code:

wait(mutex)

wait(full)

consume item

signal(empty)

signal(mutex)

Identify the synchronization bug causing the system to hang. Explain which deadlock condition is being violated. Suggest an optimization to ensure correct synchronization without changing the producer- consumer functionality. (10 Marks)

1. Given Information

- mutex = 1
- empty = 0
- full = 10
- Buffer size = **10**

The producer and consumer both acquire mutex **before** checking the availability semaphore.

2. Identify the Bug

The main synchronization bug is the **order of semaphore operations**.

Producer:

wait(mutex)

wait(empty)

produce item

signal(full)

signal(mutex)

Consumer:

wait(mutex)

wait(full)

consume item

signal(empty)

signal(mutex)

The problem occurs because a process acquires mutex **before** checking whether it can proceed.

3. How the Deadlock Occurs

Initially:

mutex = 1

empty = 0

full = 10

Since empty = 0, the buffer is **full**. Therefore, the producer cannot produce another item.

Suppose the producer executes:

wait(mutex)

Now:

mutex = 0

The producer then executes:

wait(empty)

But empty = 0, so the producer becomes **blocked while holding mutex**.

Now a consumer tries to execute:

wait(mutex)

But mutex = 0, so the consumer also becomes blocked.

Therefore:

Producer → holds mutex → waits for empty

Consumer → waits for mutex

The producer cannot release mutex because it is blocked on empty, and the consumer cannot acquire mutex to consume an item and signal empty.

4. Circular Wait

The dependency becomes:

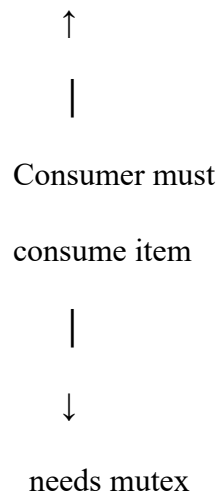
Producer

|

|— holds mutex

|

|— waits for empty



This creates a **deadlock**.

5. Deadlock Condition

The violated/created condition is **Hold and Wait**.

The producer is:

- **Holding mutex**
- **Waiting for empty**

At the same time, the consumer needs mutex to make progress and signal empty.

Thus, the producer holds one resource while waiting for another resource.

This can lead to **circular waiting** as well.

6. Correct Synchronization Order

The solution is to check the counting semaphore **before acquiring the mutex**.

Correct Producer

wait(empty)

wait(mutex)

produce item

signal(mutex)

signal(full)

Correct Consumer

wait(full)

wait(mutex)

consume item

signal(mutex)

signal(empty)

7. Why This Fix Works

When the buffer is full:

empty = 0

The producer executes:

wait(empty)

and blocks **without holding mutex**.

Therefore, the consumer can acquire mutex, consume an item, and execute:

signal(empty)

The producer is then awakened and can continue.

Similarly, when the buffer is empty:

full = 0

the consumer waits on full **without holding mutex**, allowing the producer to acquire mutex and produce an item.

8. Optimisation

Use the standard semaphore order:

Producer:

$\text{wait}(\text{empty}) \rightarrow \text{wait}(\text{mutex}) \rightarrow \text{produce} \rightarrow \text{signal}(\text{mutex}) \rightarrow \text{signal}(\text{full})$

Consumer:

$\text{wait}(\text{full}) \rightarrow \text{wait}(\text{mutex}) \rightarrow \text{consume} \rightarrow \text{signal}(\text{mutex}) \rightarrow \text{signal}(\text{empty})$

This ensures that a process never holds the mutex while waiting for buffer space or an available item.

9. Expected Result

Before:

$\text{wait}(\text{mutex}) \rightarrow \text{wait}(\text{empty/full})$

↓

May block while holding mutex

↓

Deadlock

After:

$\text{wait}(\text{empty/full}) \rightarrow \text{wait}(\text{mutex})$

↓

Only acquire mutex when operation can proceed

↓

No deadlock

10. Conclusion

The synchronization bug is that **mutex is acquired before empty or full**. When the buffer is full, the producer can hold mutex while waiting for empty, preventing the consumer from acquiring mutex to free a buffer slot. This creates a deadlock through **hold-and-wait and circular waiting**.

The correct solution is to **wait on empty/full first and then acquire mutex**. This preserves the producer-consumer functionality while ensuring correct synchronization and preventing deadlock.

Q7. Thrashing Detection and Optimization

A virtual memory system reports the following statistics:

Number of processes: 50

Average page fault rate: 250 faults/sec

CPU Utilization: 18%

Disk Utilization: 95%

Free Frames Available: 2%

The operating system suspects thrashing. Debug the root cause of the performance degradation.

Explain the relationship between page faults and CPU utilization.

1. Given Information

- Number of processes = **50**
- Average page fault rate = **250 faults/sec**
- CPU Utilization = **18%**
- Disk Utilization = **95%**
- Free Frames = **2%**
- Suspected problem = **Thrashing**

2. Identify the Root Cause

The system is experiencing **thrashing**.

Thrashing occurs when the operating system spends most of its time **handling page faults and swapping pages between disk and physical memory**, rather than executing processes.

The main evidence is:

- **250 page faults/sec** → very high
- **CPU utilization = 18%** → very low

- **Disk utilization = 95%** → extremely high
- **Free frames = 2%** → very little physical memory available
- **50 processes** → high degree of multiprogramming

Therefore, the processes do not have enough frames to hold their active working sets.

3. What Happens During Thrashing?

The execution cycle becomes:

Process needs page



Page not in RAM



Page fault



Disk I/O



Load page into RAM



Another required page is removed



Another page fault



Repeated continuously

The system spends most of its time performing **disk I/O**.

4. Relationship Between Page Faults and CPU Utilization

There is a strong inverse relationship in this situation.

When the **page fault rate increases**, processes frequently stop executing and wait for pages to be loaded from disk.

Therefore:

High Page Faults



More Disk I/O



Processes Wait



Less CPU Execution



Low CPU Utilization

Here, the page fault rate is **250 faults/sec**, while CPU utilization is only **18%**.

This means the CPU is frequently idle because processes are waiting for memory pages.

5. Why Disk Utilization Is 95%

Every page fault may require a disk access to retrieve the required page.

Because page faults are occurring continuously, the disk becomes heavily occupied.

Page Faults ↑



Disk Requests ↑



Disk Utilization = 95%

Thus, the high disk utilization is strong evidence of thrashing.

6. Role of Free Frames

Only **2% of frames are free**.

This means physical memory is almost completely occupied.

When a process needs a new page, the OS may have to remove another page from memory.

If the removed page is needed again soon, another page fault occurs.

This creates a continuous cycle of **page replacement and page faults**.

7. Effect of 50 Processes

Having **50 processes** increases the degree of multiprogramming.

Each process requires a certain number of pages for its **working set**.

If too many processes are active simultaneously, the available frames are divided among them, giving each process too few frames.

This increases the page fault rate and causes thrashing.

8. Optimisation

The main solution is to **reduce the degree of multiprogramming**.

Temporarily suspend or swap out some processes and allocate more frames to the remaining active processes.

For example:

50 Processes



Reduce active processes



Fewer processes in memory



More frames per process



Fewer page faults



Less disk I/O



Higher CPU utilization

Another useful technique is **working-set management**, where the OS ensures that each active process has enough frames for its current working set.

9. Expected Improvement

Metric	Current Condition	After Optimization
Page faults	250/sec	Decrease
CPU utilization	18%	Increase
Disk utilization	95%	Decrease
Free frames	2%	Increase
Active processes	50	Reduced if necessary
Performance	Poor	Improved

10. Conclusion

The system is suffering from **thrashing** because **50 processes are competing for limited physical frames**. The evidence is the **very high page fault rate (250/sec)**, **very low CPU utilization (18%)**, **very high disk utilization (95%)**, and **only 2% free frames**.

Frequent page faults force processes to wait for disk I/O, leaving the CPU underutilized. The best optimization is to **reduce the degree of multiprogramming and use working-set/page-frame management**. This gives active processes enough memory, reduces page faults and disk activity, and improves CPU utilization.

Q8. File System Performance Debugging

A database server experiences slow file access times. System logs indicate:

Average file size: 500 MB

Disk seeks per request: 120

Disk utilization: 92%

Cache hit ratio: 15%

Identify the file system bottleneck affecting performance. Explain why disk access latency is increasing. Suggest two optimizations to improve file access efficiency.

1. Given Information

- Average file size = **500 MB**
- Disk seeks per request = **120**
- Disk utilization = **92%**
- Cache hit ratio = **15%**

2. Identify the Bottleneck

The main bottleneck is **excessive disk seeking**.

There are **120 disk seeks per request**, which is very high. Each seek requires the disk head to move to the required location, increasing the total access time.

The **92% disk utilization** also indicates that the disk is heavily loaded.

The **15% cache hit ratio** is very low, meaning most file requests cannot be served from memory/cache and must access the disk.

3. Why Disk Access Latency Is Increasing

The access time can be viewed as:

Disk access time $\approx$ Seek time + Rotational latency + Data transfer time

With 120 seeks per request:

Request



Seek



Read data



Seek again



Read data



... repeated many times

Each additional seek adds latency.

Because disk utilization is already **92%**, many requests are competing for the disk. This creates a queue of pending I/O requests and further increases waiting time.

4. Effect of Low Cache Hit Ratio

Only **15%** of requests are satisfied by the cache.

Therefore:

85% of requests require disk access.

For example, out of 100 requests:

- 15 → served from cache
- 85 → require disk access

This significantly increases disk workload.

5. Main Causes

The performance degradation is caused by:

1. **High number of disk seeks (120/request).**

2. **Low cache hit ratio (15%).**
3. **High disk utilization (92%).**
4. Large files of around **500 MB** may require significant data transfer and can increase fragmentation-related access if their blocks are scattered.

6. Optimisation 1 – Increase Cache Efficiency

Increase the file-system/database cache and improve caching policies.

Frequently accessed data can be kept in memory so that future requests do not require disk access.

Before:

Request → Cache miss → Disk → Data

After:

Request → Cache hit → Data

Improving the cache hit ratio will reduce the number of disk requests and therefore reduce disk utilization and latency.

7. Optimisation 2 – Reduce Disk Seeks

Use **file defragmentation and better data placement** so that related blocks are stored closer together.

For database systems, also consider:

- Sequential file access
- Proper indexing
- Data clustering
- Sequential I/O instead of random I/O

For example:

Before:

Block 1 → Block 500 → Block 20 → Block 800

Many seeks

After:

Block 1 → Block 2 → Block 3 → Block 4

Fewer seeks

This reduces seek overhead and improves file access speed.

8. Expected Improvement

Metric	Current	After Optimisation
File size	500 MB	Same
Seeks/request	120	Reduced
Disk utilization	92%	Reduced
Cache hit ratio	15%	Increased
Access latency	High	Lower

9. Overall Effect

Better caching

+

Fewer disk seeks

↓

Fewer disk I/O operations

↓

Lower disk queue

↓

Lower access latency



Better file-system performance

10. Conclusion

The major file-system bottleneck is **excessive disk seeking combined with a very low cache hit ratio**. The **120 seeks per request** and **92% disk utilization** cause high disk latency, while the **15% cache hit ratio** forces most requests to access the disk.

The two main optimisations are:

1. **Increase cache efficiency** to serve more requests from memory.
2. **Reduce disk seeks** through better data placement, defragmentation, indexing, clustering, and sequential I/O.

These changes reduce disk workload and significantly improve file access performance.

Q9. Thread Scheduling Debugging

A web server creates one thread per client request. During peak hours:

Active Threads: 5,000

CPU Utilization: 60%

Memory Usage: 92%

Average Response Time: 12 seconds

Scheduler logs show frequent thread creation and termination. Identify the primary issue affecting server performance. Explain how thread management contributes to the delay. Recommend two optimizations to improve scalability and response time.

1. Given Information

- Active threads = **5,000**
- CPU utilization = **60%**
- Memory usage = **92%**
- Average response time = **12 seconds**
- Scheduler shows **frequent thread creation and termination**
- Model = **One thread per client request**

2. Primary Problem

The main problem is **excessive thread creation and termination**.

Creating a new thread for every client request is expensive. Each thread requires:

- Thread Control Block (TCB)
- Stack memory
- CPU time for creation
- Scheduling resources
- Context-switching overhead

With **5,000 active threads**, memory consumption is already very high at **92%**.

3. How Thread Management Causes Delay

For every incoming request, the server performs:

Client Request



Create Thread



Allocate Stack/Resources



Schedule Thread



Process Request



Terminate Thread



Release Resources

During peak hours, thousands of these operations occur repeatedly.

The operating system must continuously create, schedule, switch between, and terminate threads.

This causes **thread-management overhead**.

4. Effect of High Memory Usage

Memory usage is **92%**, which is very high.

Every thread requires stack space and other kernel resources. Therefore:

More Threads



More Memory Required



Memory Usage ↑



Memory Pressure



Performance ↓

High memory pressure can also increase paging and reduce the memory available for actual application data.

5. Effect on Response Time

The average response time is **12 seconds**, which indicates significant delay.

The CPU is only **60% utilized**, so the CPU itself is not necessarily the primary bottleneck.

Instead, the server is spending significant resources on:

- Thread creation
- Thread termination
- Scheduling
- Context switching
- Memory management

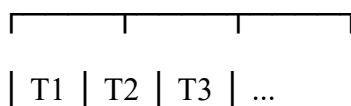
Therefore, requests spend more time waiting before completion.

6. Optimisation 1 – Thread Pool

Use a **thread pool** instead of creating a new thread for every request.

A fixed number of worker threads can be created at startup.

Thread Pool





Request Queue



Client Requests

When a request arrives, an available thread processes it and then returns to the pool.

This eliminates repeated thread creation and termination.

Benefits:

- Lower thread creation overhead
- Lower memory consumption
- Fewer context switches
- Better response time
- Improved scalability

7. Optimisation 2 – Limit and Tune Concurrency

Set a reasonable maximum number of worker threads and use a **request queue** for excess requests.

For example:

5000 Requests



Request Queue



Controlled Worker Threads



CPU / I/O Processing

Instead of allowing thousands of threads to execute simultaneously, the server controls concurrency according to available CPU and memory.

For I/O-heavy workloads, asynchronous/non-blocking I/O can also reduce the number of threads required.

8. Before Optimisation

Request → Create Thread → Execute → Terminate

Request → Create Thread → Execute → Terminate

Request → Create Thread → Execute → Terminate

↓

Huge thread overhead

9. After Optimisation

Requests

↓

Queue

↓

Thread Pool

↓

Worker Thread

↓

Return to Pool

Threads are reused instead of repeatedly created and destroyed.

10. Conclusion

The primary performance issue is **thread explosion caused by creating one thread for every client request**. The **5,000 active threads and 92% memory usage** create significant scheduling and memory-management overhead, contributing to the **12-second response time**.

The two recommended optimisations are:

1. **Use a thread pool** to reuse a controlled number of worker threads.
2. **Limit concurrency and use request queues/asynchronous I/O** to prevent excessive thread creation.

These changes reduce thread-management overhead, lower memory usage, improve scalability, and reduce response time.

Q10. Disk Scheduling Optimization Problem

A disk scheduling log shows the following request queue:

98, 183, 37, 122, 14, 124, 65, 67

Current Head Position: 53

The system uses FCFS scheduling and reports:

Average seek time: Very High

Throughput: Low

User complaints regarding slow responseDebug why FCFS is causing poor performance in this scenario. Compare FCFS with one suitable disk scheduling algorithm for this workload. Propose an optimization to improve seek time and overall throughput.

Q10. Disk Scheduling Optimization – 10 Marks

1. Given Information

Request queue:

98, 183, 37, 122, 14, 124, 65, 67

Initial head position: 53

Current algorithm: FCFS

2. Why FCFS Causes Poor Performance

FCFS (**First-Come, First-Served**) services disk requests strictly in the order they arrive.

Starting from position **53**:

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

The head makes large movements between consecutive requests.

3. Seek Movement Calculation

Movement	Distance
53 → 98	45

Movement	Distance
98 → 183	85
183 → 37	146
37 → 122	85
122 → 14	108
14 → 124	110
124 → 65	59
65 → 67	2
Total	640 cylinders

Average head movement:

$$640 / 8 = 80 \text{ cylinders/request}$$

Therefore, FCFS produces a high average seek distance.

4. Why Performance Is Poor

FCFS does not consider the physical location of requests.

For example:

183 → 37

requires a movement of **146 cylinders**.

Similarly:

14 → 124

requires **110 cylinders**.

These large movements increase:

- Seek time
- Disk access latency

- Request waiting time
- Response time

Therefore, throughput becomes low.

5. Suitable Algorithm – SSTF

A suitable alternative is **SSTF (Shortest Seek Time First)**.

SSTF selects the request that is **closest to the current head position**.

Starting from 53:

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

The movements are:

Movement	Distance
53 → 65	12
65 → 67	2
67 → 37	30
37 → 14	23
14 → 98	84
98 → 122	24
122 → 124	2
124 → 183	59
Total	236 cylinders

Average movement:

$$236 / 8 = 29.5 \text{ cylinders/request}$$

6. FCFS vs SSTF

Feature	FCFS	SSTF
Total head movement	640	236
Average movement	80	29.5
Request ordering	Arrival order	Nearest request
Seek time	High	Lower
Throughput	Low	Higher
Starvation	No	Possible
Performance	Poor for random requests	Better for this workload

SSTF reduces the total head movement from **640 to 236 cylinders**, which is a significant improvement for this particular request queue.

7. Optimization

Replace **FCFS with SSTF** for this workload.

The scheduler should select the request requiring the shortest head movement.

FCFS:

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Large movements

SSTF:

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Smaller movements

This reduces seek time and allows more requests to be completed in the same amount of time.

8. Limitation of SSTF

Although SSTF performs better here, it can cause **starvation**.

If new requests continuously arrive near the current head position, requests far away may wait for a long time.

For a busy production system, **SCAN (Elevator algorithm)** can be another good choice because it provides more predictable service and reduces starvation.

9. Expected Result

SSTF



Less head movement



Lower seek time



Lower request waiting time



Higher throughput



Better response time

10. Conclusion

FCFS performs poorly because it services requests without considering their physical disk locations. For the given queue, FCFS requires **640 cylinders** of total head movement, giving an average of **80 cylinders/request**.

SSTF reduces this to **236 cylinders**, or **29.5 cylinders/request**, by selecting the closest request first. Therefore, replacing FCFS with **SSTF** can significantly improve seek time, throughput, and response time for this workload. For long-running systems where fairness is important, **SCAN** can also be considered.